CS4475 Fall 2026 Lab Report
Maria Pedroza October 14, 2026
mipedroza@miners.utep.edu

PA2: Priority-based Scheduler for xv6

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

Implementation Description:

To make the getprocs() system call work, I had to connect the user-level ps command to the kernel's process table. Here is how I built it step-by-step:

- **Shared Data Structures:** I defined the process states (enum procstate) and created struct pstat inside kernel/pstat.h. This ensures that both the kernel and the user program are looking at the exact same data layout.
- **System Call Wiring:** I registered the new system call (SYS_getprocs) across the standard xv6 routing files (syscall.h, syscall.c, user.h, and usys.pl). This creates the bridge so the user space can call the kernel function.
- **Fetching the Argument:** In kernel/sysproc.c, I wrote a wrapper function called sys_getprocs() that grabs the memory address the user provided.
- **Gathering the Data:** The main logic lives in kernel/proc.c. My getprocs() function loops through the operating system's process table. For every active process, it locks the process to prevent changes, copies its details (like PID, state, size, parent PID, and name) into a temporary struct, and then unlocks it.
- **Sending Data to User Space:** I used the kernel's copyout() function to safely push this gathered data back to the user's memory array.
- **Compiling the User Program:** Finally, I added the provided ps.c to the Makefile. I also updated a minor formatting issue in its printf statement by changing a %l to %d and casting the size variable to an integer so it would compile cleanly.

Files changed:

1. kernel/pstat.h
2. kernel/proc.h
3. kernel/syscall.h
4. kernel/syscall.c
5. kernel/sysproc.c
6. kernel/proc.c
7. kernel/defs.h
8. user/user.h
9. user/usys.pl
10. user/ps.c (Added)
11. Makefile

Results:

```
init: starting sh
$ ps
pid     state           size    ppid    name
1       sleeping        16384   0       init
2       sleeping        20480   1       sh
3       running         16384   2       ps
$
```

Task 2. (15 pts) Modify the provided ps command to print the priority of each process.

To get the ps command to show process priorities, I had to update the data pipeline so getprocs() would fetch the new information.

1. Updated the Shared Form: First, I added int priority; to struct pstat in kernel/pstat.h. This ensures that when getprocs() sends data to the user space, it has a designated slot for the priority value.
2. Copied the Data: Inside the actual getprocs() function in kernel/proc.c, I had to add a line of code inside the process-scanning loop: st.priority = p->priority;. This tells the system call to explicitly grab the priority from the kernel's process vault and add it to our struct before sliding it over to the user.

Changes made to ps.c:

- I modified the header printf statement to include a priority column at the end.
- I updated the data printf statement inside the loop to include a %d format specifier and pull uproc[i].priority so it prints the value to the screen.

```
$ testpriority
Setting priority to 25...
Priority set! Running ps to verify...
pid     state           size    ppid    name    priority
1       sleeping        16384   0       init    0
2       sleeping        20480   1       sh      0
3       running         16384   2       ps      25
```

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

Timestamping the Arrival: Whenever a process enters the ready line (transitions from another state into RUNNABLE), the kernel stamps it with the current clock time (p->readytime = ticks;). This happens across the five places where processes become runnable: userinit(), fork(), yield(), wakeup(), and kill().

The Math: In ps.c, the user program checks the current time using uptime() and computes:

$$\text{age} = \text{current_time} - \text{readytime}$$

State Filter: This calculation runs only when a process's state is RUNNABLE. If a process is SLEEPING or actively RUNNING, its age defaults to 0 because it is not waiting in the ready queue.

Results:

```
init: starting sh
$ testage
Starting CPU-bound children to test age...
Running ps to check runnable age:
pid     state           size    ppid    name    priority        age
1       sleeping        16384   0       init    0               0
2       sleeping        20480   1       sh      0               0
3       running         16384   2       ps      0               0
4       runnable        16384   3       testage 0               1
5       runnable        16384   3       testage 0               0
$ ps
pid     state           size    ppid    name    priority        age
1       sleeping        16384   0       init    0               0
2       sleeping        20480   1       sh      0               0
6       running         16384   2       ps      0               0
```

Task 4. (15 pts) Implement a priority-based scheduler.

- **kernel/param.h**: I added compile-time scheduling constants (#define RR 0, #define PRIORITY 1, #define SCHEDULER PRIORITY)
    - Purpose: Enables switching between the default round-robin algorithm and the priority-based scheduler at compile time without rewriting core kernel logic.
- **kernel/proc.c (scheduler())**: I implemented an #if SCHEDULER == PRIORITY conditional block containing a dual-pass priority selection algorithm.
    - Purpose: Acts like a traffic cop for the CPU. Instead of taking turns in a simple circle, it looks at all processes waiting in line, finds the one with the highest priority number, and lets that one run first. If there is a tie, it picks the first one it sees. If nothing is waiting, it puts the CPU to sleep to save power.
- **user/testpriority.c**: created a test program spawning three child processes with priorities 10, 20, and 30.
    - Purpose: Proves that higher-priority processes finish first. It starts three tasks with priorities 10, 20, and 30, holds them at a starting line using a pipe, and lets them go at the exact same moment.
- **user/testtie.c**: created a test program spawning three child processes sharing an identical priority of 25.
    - Purpose: Proves how the computer handles ties. It launches three tasks with the exact same priority (25) to make sure the scheduler picks them one by one without freezing or crashing.
- **Makefile:** Added $U/_testtie\ to the UPROGS target list.
    - Purpose**:** Tells the compiler to include testtie so we can run it as a command in the xv6 terminal. testpriority.c had been created before and just modified for new testing, so it was already in the Makefile UPROGS list.

What Was Learned

- **VIP Access for the CPU:** With strict priority scheduling, the CPU gives 100% of its attention to the highest-priority task until that task either finishes, needs to wait, or gives up the CPU.
- **Why One Core Matters for Testing:** If your computer has multiple CPU cores running at the same time, it can cheat by running the low-priority tasks on another core. To prove our priority rules work, we had to force the computer to use just one CPU core (CPUS=1).
- **Managing the Locks:** Whenever the scheduler checks or switches tasks, it has to lock the process table carefully so two CPUs or interrupts don't mess up the task's state midway.

Difficulties and Solutions

- **The Head Start Problem:**
  - *Difficulty:* In our first test, the low-priority task finished before the high-priority tasks even started. This happened because the parent process created Task 1 and let it run immediately while the parent was still busy setting up Task 2 and Task 3.
  - *Solution:* We added a "starting line" using an xv6 pipe. All tasks set their priority and then freeze, waiting for a signal. Once all three are ready, the parent sends the signal, so they all start competing at the exact same millisecond.
- **Multiple CPUs Messing Up the Results:**
  - *Difficulty:* xv6 runs on 3 CPU cores by default. Even though Task 30 had the highest priority, Tasks 20 and 10 were still finishing because the other two cores were running them simultaneously.
  - *Solution:* We launched xv6 with CPUS=1 qemu. Putting everything onto a single CPU forced the scheduler to follow the priority order strictly.

Strict Priority Verification (testpriority):

```
init: starting sh
$ testpriority
Starting synchronized priority verification...
Child with priority 30 finished.
Child with priority 20 finished.
Child with priority 10 finished.
All priority tests completed.
```

Tie-Breaking verification (testtie):

```
$ testtie
Starting tie-breaking priority verification (priority 25)...
Child 1 (pid 4, pri 25) finished.
Child 2 (pid 5, pri 25) finished.
Child 3 (pid 6, pri 25) finished.
All tied tests completed.
```

Task 5. (15 pts) Add aging to your priority-based scheduler.

1. Aging Policy Explanation

- **The Problem:** In a strict priority system, a low-priority process might never get to run if higher-priority processes keep arriving (this is called starvation).
- **The Policy Formula:** We used the formula:

$$\text{effective priority} = \text{priority} + \text{age}$$

where $\text{age} = \text{ticks} - \text{readytime}$.

- **How It Works:** Every clock tick a process spends waiting in line (RUNNABLE), its age increases by 1. That wait time gets added directly to its priority score. Eventually, even a low-priority task becomes important enough to beat high-priority tasks and get a turn on the CPU.
- **The Reset:** Once the waiting task gets scheduled and uses its CPU slice, it yields back to the ready queue. Its readytime resets to the current clock tick, which drops its age back to 0. This ensures high-priority jobs still get priority overall, while low-priority jobs are guaranteed progress without getting stuck forever.

2. Test Results & Benefits of Aging

Test 1: Verification with Background Matrix Jobs (pexec and matmul) We tested aging by running a low-priority matrix job (priority 0) and a high-priority matrix job (priority 5) simultaneously, while checking process status using ps:

```
$ pexec 5 matmul 5 &; matmul 10 &; pexec 10 ps
pid   state      size   ppid   name    priority   age
1     sleeping   16384  0      init    0          0
2     sleeping   20480  1      sh      0          0
3     sleeping   16384  2      pexec   10         0
8     running    16384  3      ps      10         0
5     sleeping   16384  1      pexec   5          0
7     runnable   16384  1      matmul  0          0
9     runnable   16384  5      pexec   5          0
Time: 173 ticks
```

- **Benefit Observed:** In pure priority mode, the priority 0 process would be completely blocked until priority 5 finished. With aging, both tasks made active progress, and the priority 5 job finished at 173 ticks without freezing the rest of the system.

Test 2: Benchmark Evaluation (testbench) We created an automated benchmark program (testbench) running three tasks with priorities 0, 10, and 20 released at the exact same moment on a single CPU core. We ran this under both pure priority and aging to measure the difference:

- **Strict Priority (SCHEDULER PRIORITY)**:

```
init: starting sh
$ testbench
Starting scheduler benchmark with 3 processes...

--- Benchmark Results ---
Job 0 (Priority 0): Response Time = 91 ticks, Turnaround Time = 92 ticks
Job 1 (Priority 10): Response Time = 49 ticks, Turnaround Time = 92 ticks
Job 2 (Priority 20): Response Time = 0 ticks, Turnaround Time = 136 ticks
Average Response Time: 46 ticks
Average Turnaround Time: 106 ticks
$
```

- **Priority with Aging (SCHEDULER AGING)**:

```
init: starting sh
$ testbench
Starting scheduler benchmark with 3 processes...

--- Benchmark Results ---
Job 0 (Priority 0): Response Time = 20 ticks, Turnaround Time = 58 ticks
Job 1 (Priority 10): Response Time = 10 ticks, Turnaround Time = 92 ticks
Job 2 (Priority 20): Response Time = 0 ticks, Turnaround Time = 128 ticks
Average Response Time: 10 ticks
Average Turnaround Time: 92 ticks
$
```

- **The Takeaway:** Aging cut the average response time from 46 ticks down to 10 ticks (a 78% improvement). The lowest-priority job went from waiting 91 ticks down to just 20 ticks before getting its first turn on the CPU.

3. What Was Learned

- **Preventing Starvation:** Pure priority systems can be unfair to background tasks. Aging acts as an automatic fairness mechanism so every process eventually gets served.
- **Response Time vs. Turnaround Time:** We learned that aging drastically reduces response time (how long a task waits before it starts) because tasks don't have to wait for every higher-priority job to finish completely before getting a turn.
- **State Lifecycle Matters:** Aging only works if you correctly track when a process becomes ready to run (RUNNABLE) versus when it is actively running (RUNNING). Resetting the wait timer when a task yield is essential, so it doesn't hoard the CPU permanently.

4. Difficulties and Solutions

- **Matrix Dimension Caused Apparent Freezing:**
  - *Difficulty:* When testing with matmul 100, the terminal appeared completely frozen with no shell prompt returning for several minutes.
  - *Solution:* Multiplying 100×100 matrices on a single emulated core in QEMU requires over a million operations. We switched to the course-recommended

smaller dimensions (matmul 5 and matmul 10), which finished in a few seconds and printed results cleanly.

- **Typo in Variable Names during Compiler Switch:**
    - *Difficulty:* When switching param.h to use AGING, the compiler threw errors about max_score being undeclared and max_priority being unused.
    - *Solution:* In kernel/proc.c, line 525 had declared max_priority instead of max_score inside the #if SCHEDULER == AGING block. Renaming the tracking variable to max_score resolved the issue and allowed xv6 to compile cleanly.

Task 6. (5 pts) Summary

What Was Learned from Doing This Assignment

1. **User Space vs. Kernel Space Isolation**
    - Operating systems enforce strict hardware memory boundaries. User programs cannot directly inspect kernel memory.
    - Passing process tables across this boundary safely requires dedicated system calls (getprocs), register argument retrieval (argaddr), and address-space translation via copyout.
2. **Process Lifecycle and State Transitions**
    - Keeping accurate time accounting requires instrumenting every single transition into RUNNABLE.
    - Setting readytime across userinit, fork, yield, wakeup, and kill reinforced how an OS schedules threads between sleeping, waiting, and actively executing.
3. **Scheduler Trade-offs: Round-Robin vs. Priority vs. Aging**
    - **Round-Robin:** Guarantees fairness but cannot favor critical or interactive workloads.
    - **Strict Priority:** Prioritizes important jobs but causes indefinite starvation for low-priority processes when workloads are continuous.
    - **Priority with Aging:** Provides the ideal balance by dynamically promoting waiting jobs based on queue latency, eliminating starvation.
4. **Quantifying Scheduler Performance**
    - Evaluating schedulers requires empirical metrics rather than just verifying task completion order.
    - Measuring **Response Time** (time until first CPU execution) versus **Turnaround Time** (total time to finish) demonstrated that aging reduced response time by **78%** without hurting system throughput.
5. **Single-Core Emulation for Scheduler Testing**
    - Modern multi-core setups mask scheduling priority rules by executing lower-priority threads simultaneously on idle cores.
    - Constraining QEMU to a single core (CPUS=1) was essential to force the scheduler to make strict, observable sequential choices.

Difficulties Encountered and Solutions

1. **Race Conditions During Multi-Process Test Creation**

- **Difficulty:** When calling fork() in a loop inside test programs, Child 1 ran immediately and finished its workload before the parent process had even finished creating Child 2 and Child 3. This produced the illusion that low-priority processes finished first.
- **Solution:** Built a synchronization pipe barrier (sync_pipe). Each child sets its priority and blocks on read(). Once all children are initialized in the process table, the parent issues a single write() to unblock all children simultaneously at the exact same tick.

2. **Apparent System Freezes from Large Computational Workloads**
   - **Difficulty:** Running matmul 100 appeared to freeze the xv6 shell without returning a prompt or finishing ticks.
   - **Solution:** A 100×100 matrix multiplication executes $1{,}000{,}000$ loop operations, which takes several minutes under software-emulated RISC-V QEMU with timer interrupts. We reduced matrix dimensions to the assignment-recommended sizes (matmul 5 and matmul 10) and used a calibrated loop in testbench.c, producing results within seconds.
3. **Lock Management in Multi-Pass Scheduling**
   - **Difficulty:** Scanning all 64 process slots in proc[] to find the highest-scoring candidate required holding process locks to prevent race conditions without deadlocking the kernel.
   - **Solution:** Structured scheduler() to inspect candidates under lock, keep only the current highest-priority candidate locked while scanning, release all non-selected processes immediately, and only context-switch (swtch) once the winner is identified.
4. **Compiler Warnings Treated as Errors (-Werror)**
   - **Difficulty:** xv6 compiles with -Werror, treating unused variables as fatal errors. Defining variables like max_priority under #if SCHEDULER == AGING failed compilation.
   - **Solution:** Scoped and aligned local variables cleanly inside the respective #if, #elif, and #else preprocessor branches in kernel/proc.c.

AI Usage & Assistance Statement
Tool Used: Google Gemini (Large Language Model)
Date of Usage: October 2026
Assignment: CS 4475 – Programming Assignment 2: Priority-based Scheduler for xv6
Description of Assistance:
Generative AI assistance was utilized throughout this assignment for conceptual understanding, implementation support, and debugging in the following capacities:

- **Architecture & Code Brainstorming:** Brainstormed scheduling logic designs, including the dual-pass process selection algorithm for strict priority scheduling, the aging formula calculation ($EffectivePriority = Priority + Age$), and the synchronization barrier design using Unix pipes for deterministic multi-process testing.
- **Debugging & Error Diagnostics:** Assisted in troubleshooting kernel and user-space issues, such as diagnosing execution hangs during matrix multiplication (matmul),

addressing variable scope/naming compiler errors (max_score vs. max_priority) under strict -Werror flags, and debugging process table lock management to avoid deadlocks.

- **Metric Formulation & Analysis:** Guided the calculation, collection, and comparative evaluation of scheduler performance metrics (Response Time and Turnaround Time).
- **Report Drafting & Conceptual Clarification:** Provided simplified, plain-English conceptual analogies (e.g., system call vault/teller mechanics) and assisted in structuring the technical explanations and reflections.

Author Verification:

All AI-generated suggestions, code snippets, and explanations were independently reviewed, compiled, tested in QEMU on Linode, and verified by the author. The author fully understands the submitted code and report contents and takes full academic responsibility for the submission.